

1. Design

The chosen converter is a DC buck converter with the following electrical structure (Figure 1):

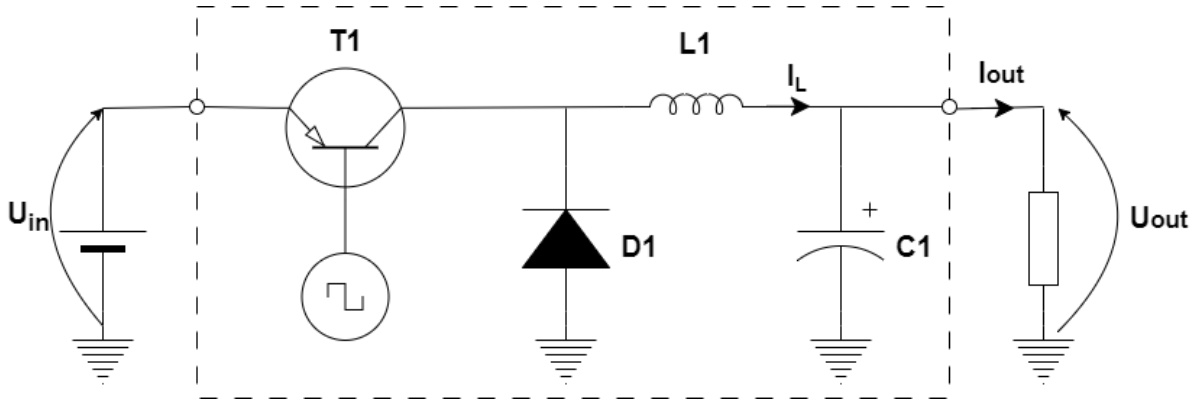


Figure 1 DC buck converter

This type of power electronic converter is working in **switching mode** to increase power and cost efficiency. In this mode, the controlling transistor is in blocking mode (no current is flowing, open circuit) or saturation mode (conduction, like a short circuit), avoiding active mode (flowing current is proportional to the base current). Switching mode is reached by applying a PWM signal on the base of the transistor, obtaining a similar waveform with higher amplitude. The mean voltage of the output is proportional with the duty cycle of the PWM signal.

On the output square wave signal filtering is applied to smooth out current and voltage ripples, obtaining a mostly continuous output voltage up to a chosen maximum current. The filter consists of an L1 inductor and C1 capacitor, with a helping D1 diode to provide path for the inductor current. The two performance indicators we use are the inductor current ripple and output voltage ripple, which shows the amplitude of the periodic variation of voltage/current.

To approximate the L1 inductor current, we have to assume that U_{out} is constant. This way the L1 inductor voltage is: $U_L = \begin{cases} U_{in} - U_{out}, & T1 \text{ is on} \\ -U_{out}, & T1 \text{ is off} \end{cases}$. The derivative of the inductor current

can be described: $\frac{dI_L}{dt} = \frac{U_L}{L} = \begin{cases} \frac{U_{in} - U_{out}}{L}, & T1 \text{ is on} \\ \frac{-U_{out}}{L}, & T1 \text{ is off} \end{cases}$. In both T1 on, and T1 off periods the

derivative is constant, so the inductor current will be a linear function (see Figure 2). This way a triangle wave is obtained, where the peak to peak current is called current ripple. To calculate the current ripple it is enough to take only one case of the derivative function, T1 on.

$$\text{The current ripple: } \Delta I_L = I_L(t_2) - I_L(t_1) = \int_{t_1}^{t_2} dI_L dt = \int_{t_1}^{t_2} \frac{U_{in} - U_{out}}{L} dt = \frac{U_{in} - U_{out}}{L} (t_2 - t_1)$$

$$\Delta I_L = \frac{U_{in} - U_{out}}{L} t_{ON} = \frac{U_{in} - U_{out}}{L} \frac{\mu}{f_s}$$

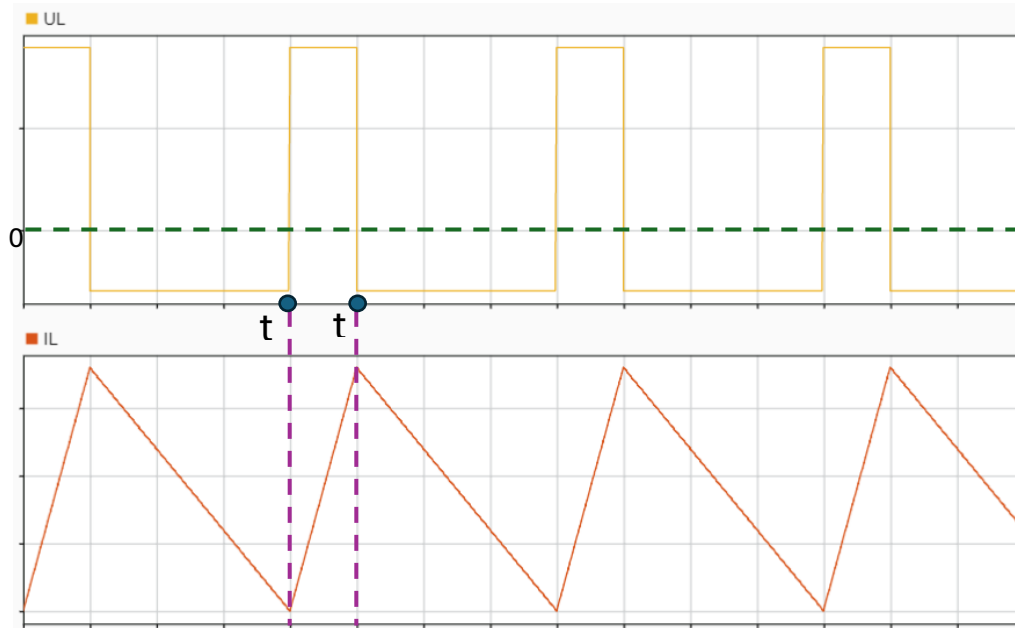


Figure 2 Inductor current

The mean value of the inductor current is I_{outmax} . Lets consider Kirkhoff's law for nodes. The capacitor current can be described as $I_C = I_L - I_{outmax}$. To obtain the capacitor voltage, the following equation needs to be used: $\frac{dU_C}{dt} = \frac{I_C}{C}$. The voltage curve is constructed with parabolas (see Figure 3), the voltage difference between peaks is called output voltage ripple.

$$\text{Voltage ripple: } \Delta U_C = U_C(t_4) - U_C(t_3) = \int_{t_3}^{t_4} dU_C dt = \int_{t_3}^{t_4} \frac{I_C}{C} dt = \frac{1}{C} \int_{t_3}^{t_4} I_C dt$$

The integral of the capacitor current between t_3 and t_4 will be the area below the function (blue triangle). $\Delta U_C = \frac{1}{C} \int_{t_3}^{t_4} I_C dt = \frac{1}{C} \frac{\Delta I_L}{2} \frac{t_p}{2} \frac{1}{2} = \frac{\Delta I_L}{8 C f_s}$

The final formulas:

The output voltage becomes $U_{out} = U_{in} * \mu$.

The inductor ripple current can be determined by: $\Delta I_L = (U_{in} - U_{out}) * \mu / (f_s * L)$

The output ripple voltage is: $\Delta U_{out} = \Delta I_L / (8 * f_s * C)$

Diode average current: $I_D = I_{outmax} * (1 - \mu)$

Transistor max current: $I_{Tmax} = \Delta I_L / 2 + I_{outmax}$

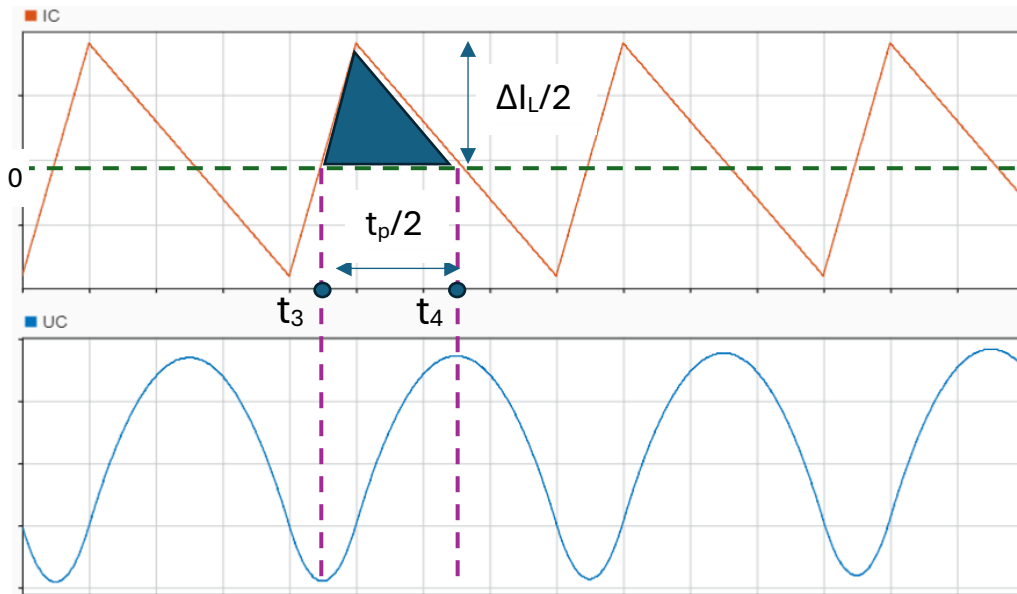


Figure 3 Capacitor voltage ripple

Task

Calculate the relevant component information to fulfill the requirements. Choose real electrical components for the circuit (mouser.com, digikey.com, ...)

Given information:

Input voltage: $U_{in} =$

Output voltage: $U_{out} =$

Maximum output current: $I_{outmax} =$

Inductor ripple current (percent of I_{outmax}): $\Delta I_L =$

Output ripple voltage: $\Delta U_{out} =$

Switching frequency: $f_s =$

To be determined:

Required duty cycle: $\mu =$

C1 capacitance: $C =$

L1 inductance: $L =$

Diode average current: $I_D =$

Transistor max current: $I_{Tmax} =$

Power Electronics

Chosen components:

C1 part number:

L1 part number:

T1 part number:

D1 part number:

Reference: <https://www.ti.com/lit/an/slva477b/slva477b.pdf>

2. Simulation

The studied electrical circuit has two modes, depending on the state of the transistor: transistor is saturated and conducting electrical current; transistor is blocked, no current flowing through it.

Model

The **model** of a physical system or phenomenon is a simplified representation of it, which is capable of catch the essential aspects of interest of the structure and behavior, from control point of view.

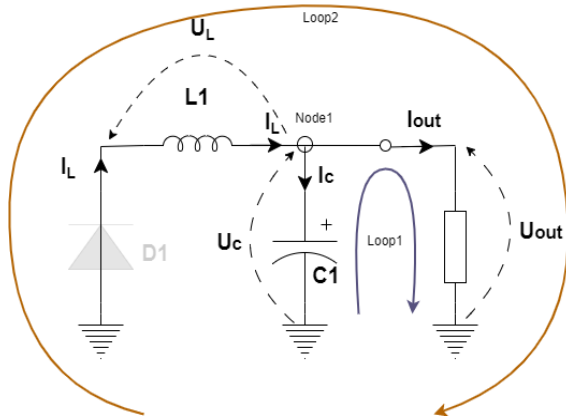
As the motivation for studying Power Electronics is the “switching mode” control, we consider some simplifying hypotheses:

- a) The commutation of the switching component is ideal, with 0 internal resistance during conduction, infinite resistance during blocking mode. Switching time, as time spent in the active region is considered as well 0.
- b) The external voltage source is considered ideal, without internal resistance.
- c) The passive elements (R, L, C) have linear characteristics, are invariant in time and without parasitic elements.

The used model type is Switched mode model, where we describe every different configuration given by switches states. In this case, we study two different scenarios.

Transistor not conducting

In this case, the path of the current flowing through the inductor is closed with the diode. The equivalent circuit looks like this.



After Kirkhoff's laws:

$$\text{Loop1: } -U_L - U_{out} = 0$$

$$\text{Loop2: } U_C - U_{out} = 0$$

$$\text{Node1: } I_L = I_C + I_{out}$$

$$-dI_L/dt \cdot L - U_{out} = 0$$

$$U_C - U_{out} = 0$$

$$I_L = dU_C/dt \cdot C + U_{out}/R$$

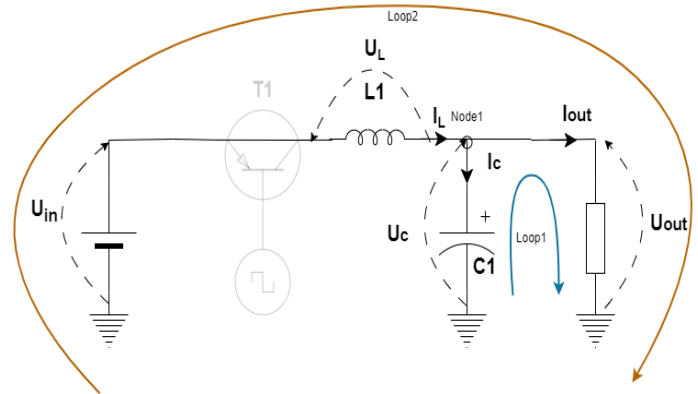
$$dI_L/dt = -U_C/L$$

$$dU_C/dt = (I_L - U_C/R)/C$$

$$U_{out} = U_C$$

Transistor conducting

In this case, the inductor current flows through the transistor, diode is remaining closed. The equivalent circuit looks like this.



After Kirkhoff's laws:

$$\text{Loop1: } U_{in} - U_L - U_{out} = 0$$

$$\text{Loop2: } U_C - U_{out} = 0$$

$$\text{Node1: } I_L = I_C + I_{out}$$

$$U_{in} - dI_L/dt \cdot L - U_{out} = 0$$

$$U_C - U_{out} = 0$$

$$I_L = dU_C/dt \cdot C + U_{out}/R$$

$$dI_L/dt = (U_{in} - U_C)/L$$

$$dU_C/dt = (I_L - U_C/R)/C$$

$$U_{out} = U_C$$

Summarizing the two systems of equation, we can obtain a generic one, introducing μ :

$$dI_L/dt = (U_{in} * \mu - U_C)/L$$

$$dU_C/dt = (I_L - U_C/R)/C$$

$$U_{out} = U_C$$

Where $\mu = 1$ shows, the transistor is conducting. $\mu = 0$ means, the transistor is blocked.

Task

Implement the system of equation described before in MATLAB-Simulink.

Input of the system is μ , output is U_{out} . Load resistance is calculated from output maximum current and output voltage.

Generate a PWM signal with the requested frequency and calculated duty cycle as input in the model.

- Using the calculated C and L values, plot out the coil current and output voltage, and measure the current and voltage ripple. Compare with the given value.
- Change C and L values to the chosen components' nominal value (the value from datasheet). How does this affect the current and voltage ripple? Plot the coil current from a) and b) on the same graph, the output voltage from a) and b) on an other graph.
- Change the input PWM signal to a constant which value is the duty cycle calculated in the Part1 of the project(between 0 and 1). How the system responds? Measure overshoot (if exists) and settling time.

3. Control

The main goal of modeling and simulation is to help the development of a control structure. In our application, we want to improve the response of the system by eliminating the overshoot and making the settling time as small as possible.

To determine the closed loop controller, we will use the Guillemin-Truxal method. To apply this method, our model of the system should be converted to a transfer function.

We must define our closed loop system performances: overshoot and settling time. From these, we can determine the transfer function of the closed loop system, using the following, well-known formulas:

Overshoot: $\sigma = 0$

Settling time: $t_r =$

Damping factor: $\xi = \frac{|\ln(\sigma)|}{\sqrt{\ln^2(\sigma) + \pi^2}} = 1$

Second order time constant: $\omega = \frac{4}{\xi * t_r} =$

The form of the closed loop system: $H_o = \frac{\omega^2}{s^2 + 2 * \omega * \xi * s + \omega^2} =$

The controller transfer function: $H_c = \frac{1}{H_p} * \frac{H_o}{1 - H_o} =$

The calculated controller is validated in closed loop simulation. **Attention:** the value of the control signal (duty cycle) should be between 0 and 1. Apply saturation block on the output of the controller. The output of the controller must be sampled with the multiple of the PWM signal period, for this we apply a Zero-Order Hold block on the control signal.

Task

Make the necessary calculations for the controller.

Set “Zero-Order Hold” block period to the (PWM period)/10. Make closed loop simulation, verify the previously defined settling time and overshoot on the output voltage graph.

Change “Zero-Order Hold” block period to PWM period. What happens with the output signal? Try to compensate for the sampling error by changing the performance indicators.

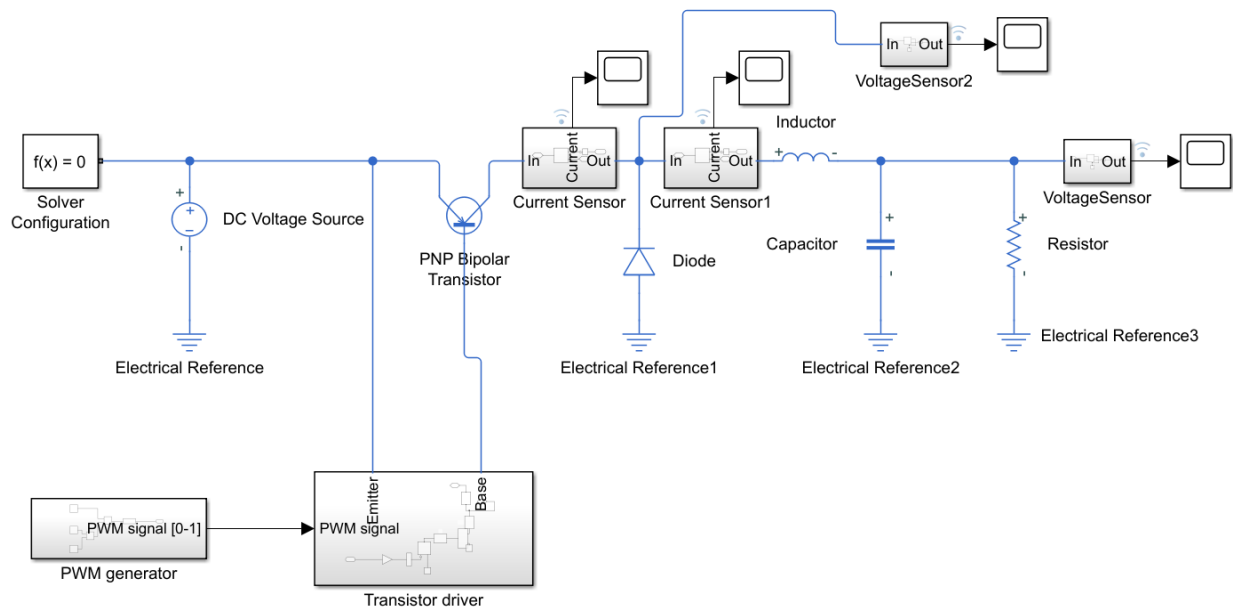
4. Advanced model and simulation

The previously discussed model is a simple representation of the real process. In this chapter we want to get closer to real behavior by using a specialized toolbox, Simscape, inside Simulink.

You must install Simscape and Simscape Electrical toolboxes.

The system topology remains the same, but we must add current and voltage sensors, and a special transistor driver.

Check out Annex 2: Simscape hints.



Task

You will create two scenarios, and you will compare the performance of the systems. In the first scenario, you will configure the components without residual elements, after you will introduce the residual properties, and tolerances. You have to introduce these changes one by one, comparing in every step the results with the a) Ideal simulation, and note down your observations. Make screenshots about the output voltage graph compared to the original.

a. Ideal simulation

We will configure the model without residual elements, to obtain comparable results with the simple model (calculated values):

DC voltage source: U_{in}

Diode: Forward voltage: 0.1 V; On resistance: 0.001 Ohm

Inductor: Inductance: L

Capacitor: Capacitance: C

Resistor: Resistance: R

- b. With residual elements (using L, C values from components with tolerance too)
 - i. Diode: Forward voltage: Forward voltage drop (datasheet)
 - ii. Capacitor: Series resistance: ESR (you can find in datasheet, or calculate with formula from datasheet, hint [Link](#))
 - iii. Inductor: Series resistance: DC resistance (datasheet DCR)
 - iv. Capacitor: Tolerance application: Apply minimum tolerance value
 - v. Inductor: Tolerance application: Apply minimum tolerance value

5. Control algorithm implementation and validation in Software in the Loop environment

In chapter 3 a control topology and continuous controller were determined in form of a transfer function. This control algorithm needs to be implemented for an Atmega328P microcontroller, which can be used to control the physical buck converter.

The closed loop controller requires two input signals: voltage setpoint and feedback voltage. As a control signal, a PWM signal is generated, with variable duty cycle. For input signals the built-in ADC is used, to measure the two analog voltages. The PWM signal is generated by a Timer/Counter in the MCU(Micro-Controller Unit). Another Timer ensures the fixed period recalculation of the control signal duty cycle.

The developed control algorithm needs to be validated before it can be used in real life scenarios, for this, multiple types of virtualized environment exist, like MiL (Model in the Loop), SiL (Software in the Loop) or HiL (Hardware in the Loop).

The simulation implemented in chapter 3 is a MiL validation environment, where the barebone control logic was tested.

In SiL concept, the control algorithm is implemented for the desired microcontroller and integrated with an additional piece of software which implements the hardware logic of the microcontroller, creating a virtual MCU. This vMCU can be used in the simulation, connected to the model of the controlled process for validation.

Task

Implement the previously calculated controller in embedded C code, generate the vMCU and validate in simulation. You have to install **MATLAB Support for MinGW-w64 C/C++/Fortran Compiler** from Add-On explorer.

- a. Transform the previously calculated H , transfer function into a discrete one, using **c2d** Matlab function and the PWM period as discretization time.
- b. Determine the recurrency equation
- c. Modify the code_controller.c file in the attached archive:
 - i. Change the OCR0A and OCR2A registers value in setup() function according to your switching frequency.
 - ii. Implement your recurrency equation between the two “**/**controler**” tag in ISR(TIMER0_COMPA_vect) interrupt function. The output of the controller needs to be saved in **command** variable.
- d. Run codeConvert.exe to integrate your code in vMCU wrapper code. To compile, run **mex vMCU_sffunction.cpp vMCU/virtualController.cpp vMCU/codeWrapper.cpp vMCU/virtualCode.cpp** code in matlab command window. Every time you are modifying your code, you have to repeat these steps.

- e. You can use the attached vMCU_model.slx as a base to test your algorithm. You have to replace the plant model with your converter model from chapter 2.
- f. Make screenshots of the output together with setpoint signal. Make screenshots of command signal. Is the model stable? The output voltage reaches the setpoint?
- g. Try to modify **filteringFactor** in setup() between 0 and 1 (0 means no input is considered – infinitely filtered, 1 means no filtering) to filter more or less the input signal. What is changing?